

Análisis de Algoritmos I

Recursión / Recursividad

L.C.C. María Janneth Rivera Reyna

Lic. En Ciencias de la Computación
Departamento de Matemáticas
Universidad de Sonora

Ciclo 2026-1

Recursión / Recursividad

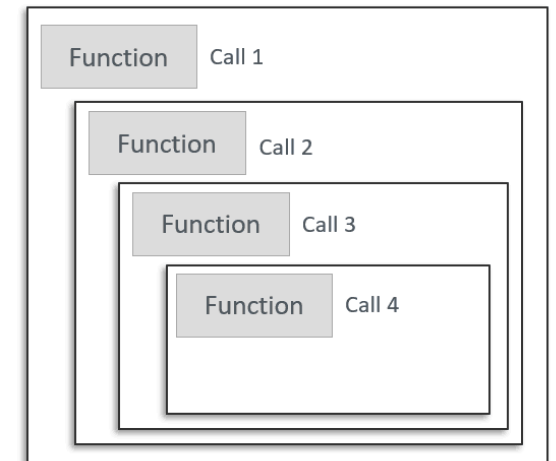
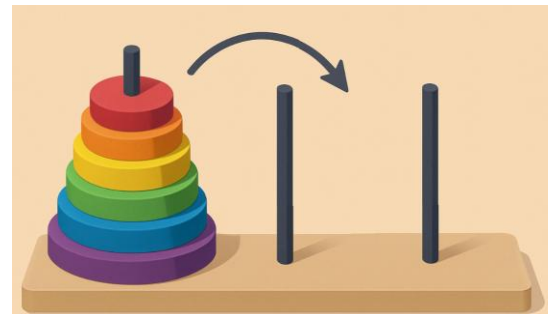
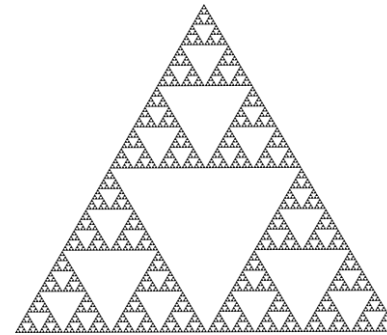


Recursión: repaso

- Técnica que nos permite diseñar un algoritmo para resolver un problema de tal modo que resuelva una instancia más pequeña del mismo problema, hasta que el problema sea tan pequeño que lo podamos resolver directamente. A esta técnica la llamamos **recursión** o **recursividad**.

Aplicaciones:

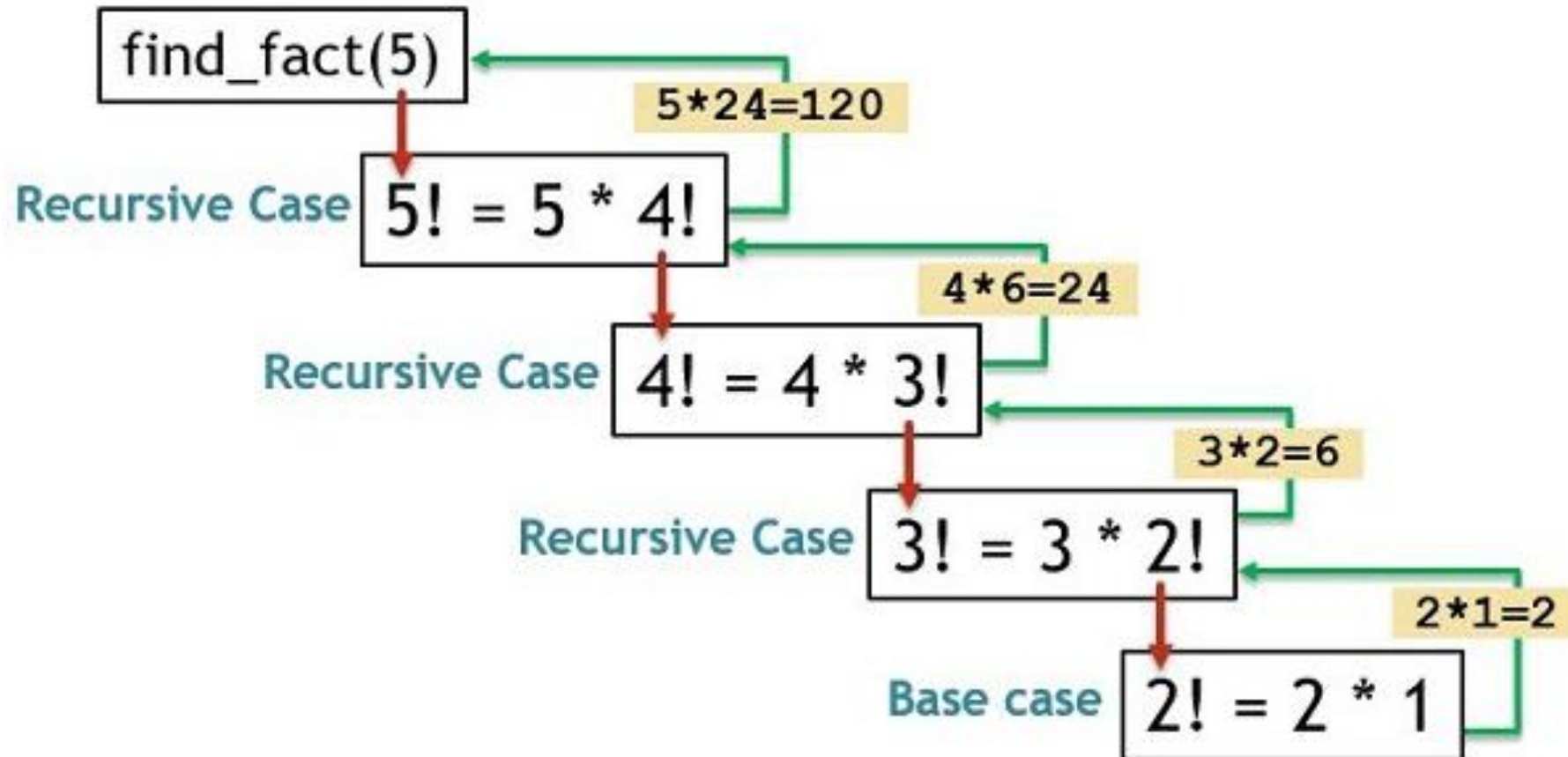
- calcular la función factorial
- determinar si una palabra es un palíndromo
- calcular potencias de un número
- dibujar un tipo de fractal
- problema de las Torres de Hanoi



Recursión: repaso

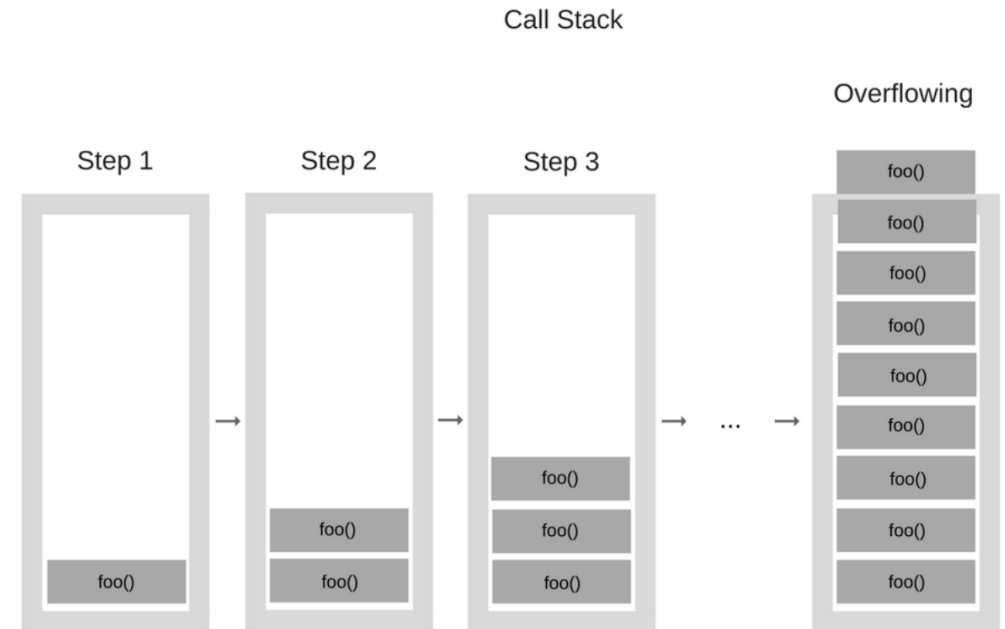
- Recursión es cuando una función se llama a sí misma.
- Una función recursiva tiene dos propiedades:
 - ✓ Una relación de recurrencia
 - ✓ Una condición de paro (caso base)
- La *relación de recurrencia* define **cómo el problema original se reduce** a una o más versiones más pequeñas de sí mismo.
- La *condición de paro* o **caso base** representa **la instancia más pequeña** posible del problema, la que ya no puede dividirse más. Será la que detenga las llamadas recursivas para evitar un **bucle infinito**.

Recursión: Ejemplo Factorial



Recursión: repaso

- Una función recursiva utiliza una **pila de llamadas (call stack)**.
- Almacena la información sobre las subrutinas activas de un programa de computadora en una estructura dinámica de datos **LIFO** (una pila).
- Cuando se consume todo el espacio asignado a la pila, ocurre el error llamado **Stack Overflow** (desbordamiento de pila).



Análisis de un algoritmo recursivo

Cuando un algoritmo es recursivo, no podemos saber directamente cuánto tardará porque el tiempo de un problema grande depende del tiempo de los problemas más chicos:

¿Cuántas llamadas recursivas hay? ¿Cuánto se divide el problema? ¿Qué operaciones adicionales hay?

Cuando un algoritmo contiene una llamada recursiva a sí mismo, a menudo podemos describir su tiempo de ejecución mediante una **ecuación de recurrencia** $T(n)$, que describe el tiempo de ejecución total en un problema de tamaño n en términos del tiempo de ejecución en entradas más pequeñas.

Por ejemplo:

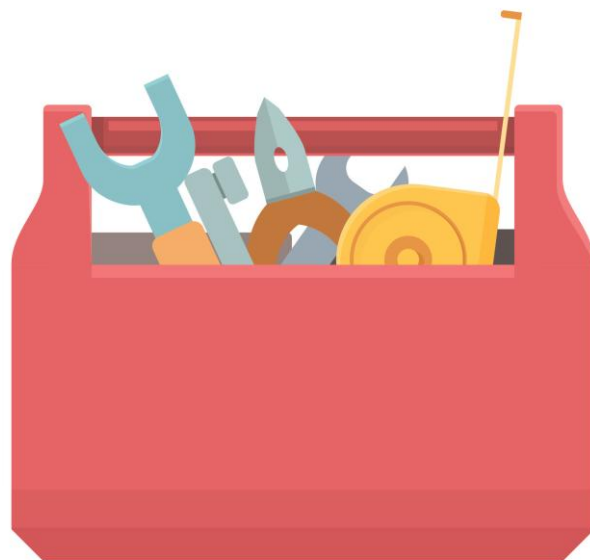
$$T(n) = T\left(\frac{n}{2}\right) + c$$

Métodos para resolver una ecuación de recurrencia $T(n)$

Dada una ecuación de recurrencia $T(n)$, para poder simplificarla con $O(T(n))$, necesitamos “romper” la recursión y convertirla en una función simple.

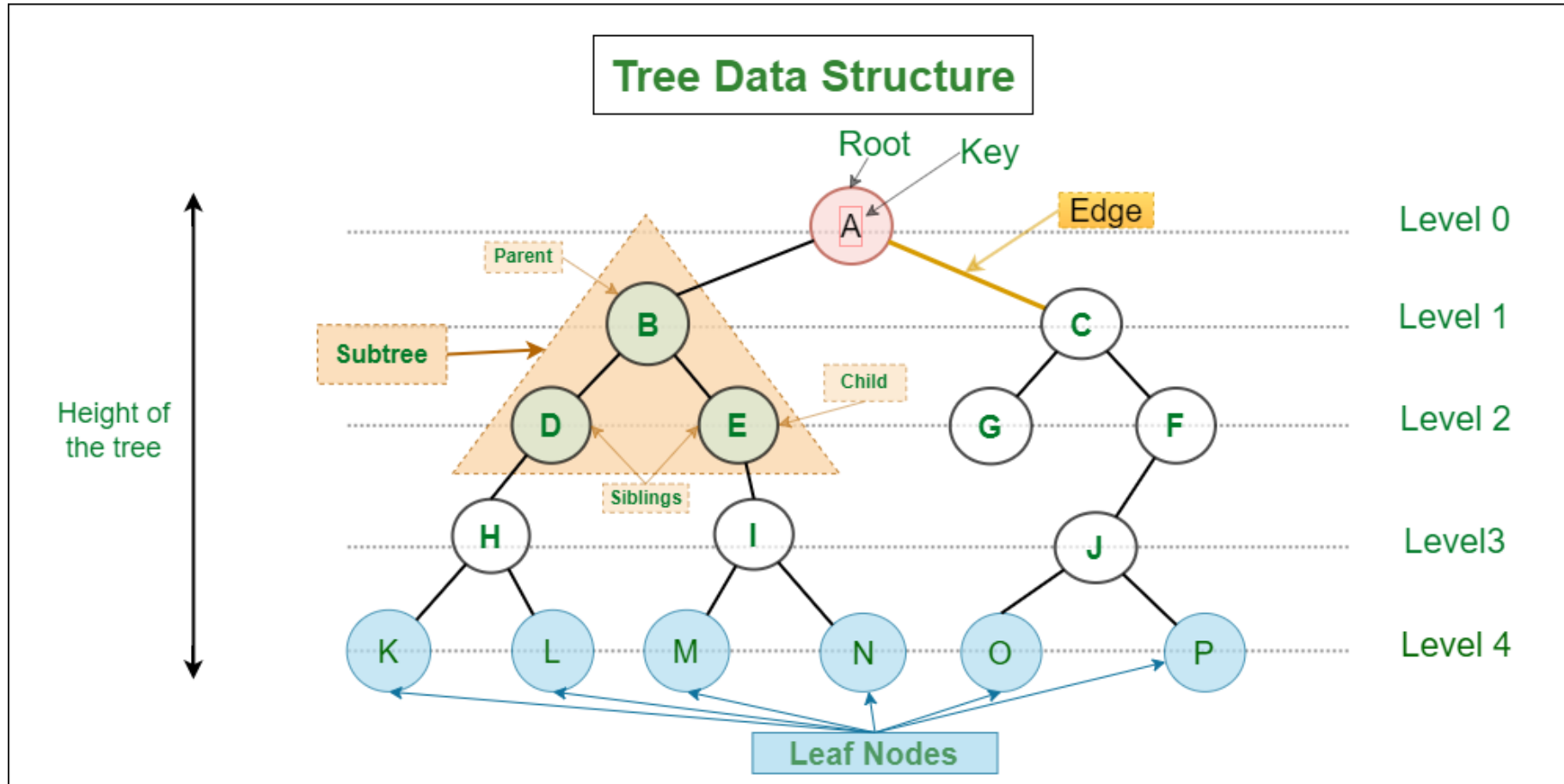
¿Cómo se resuelve una ecuación de recurrencia?

Método	¿Cuándo usarlo?	Ventaja principal	Desventaja
Sustitución (Inducción)	Cuando ya sospechas la respuesta y quieres demostrarla formalmente .	Es el único 100% riguroso matemáticamente.	Requiere buenas bases de álgebra e inducción matemática.
 Árbol de Recursión	Cuando quieres visualizar cómo se divide el trabajo.	Es el más intuitivo .	Puede volverse tedioso con ecuaciones muy complejas.
 Teorema Maestro	Para ecuaciones de la forma $T(n) = aT(n/b) + f(n)$.	Es rápido . Es una "receta" que aplicas y listo.	Solo sirve si el problema se divide en partes iguales. No sirve para $T(n) = T(n-1) + n$.



Árbol de recursión

Árbol de recursión



Árbol de recursión

- Es una **representación visual de las llamadas recursivas** realizadas durante la ejecución de un algoritmo recursivo. Nos sirve para ver qué sucede al iterar una recursión.
- En un árbol de recursión:
 - El **nodo raíz** representa el problema original de tamaño n .
 - Los **hijos de un nodo** representan los subproblemas creados por las llamadas recursivas realizadas por esa función.
 - Los **nodos hoja** representan casos base o llamadas no recursivas.

Árbol de recursión

¿Cómo calcular la complejidad de un algoritmo recursivo?

1. **Dibujar el árbol recursivo** para la relación de recurrencia dada.
2. **Calcular el costo en cada nivel** y contar el **número total de niveles** en el árbol de recursión.
3. **Sumar el costo total** de todos los niveles en el árbol recursivo.
 - a. Si el costo es igual para todos los niveles:

Costo total = costo por nivel * número de niveles

Árbol de recursión

- b. Si el costo cambia en cada nivel (crece o decrece), usamos la **Serie Geométrica**:

$$\text{Costo total} = \text{costo inicial} * \left(\frac{r^{\text{niveles}} - 1}{r - 1} \right)$$

Donde:

- **r (la razón)**: ¿Por cuánto se multiplica el costo de un nivel a otro?
- **niveles**: El número total de niveles

Nivel	Costo por nivel
0	n
1	2n
2	4n
3	8n

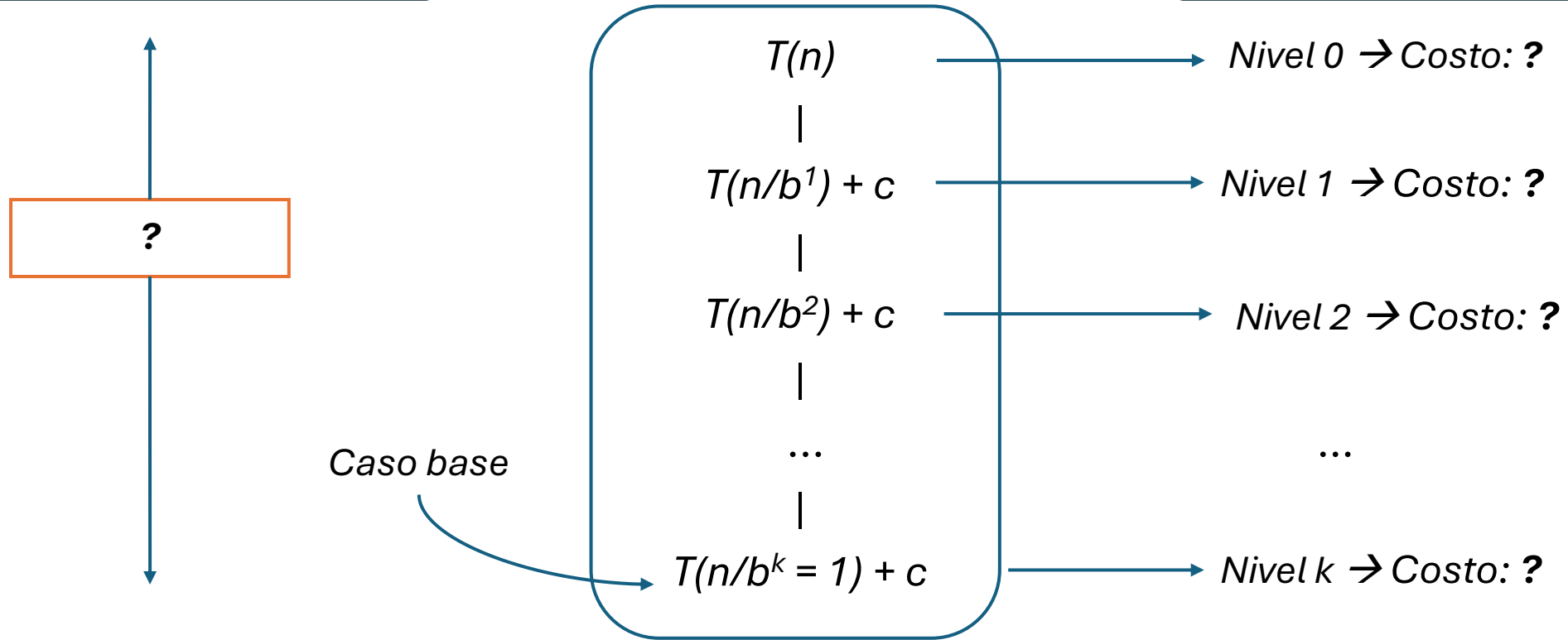
R (razón) = 2 (porque $n * 2 = 2n$ y $2n * 2 = 4n$, y así sucesivamente).

Árbol de recursión: Ejemplo genérico

$$T(n) = T\left(\frac{n}{b}\right) + c$$

¿Cuántos k niveles hay?

¿Cuánto cuesta cada nivel?



$$\begin{array}{l} \text{Costo total} = \text{Costo por nivel} * \text{número de niveles} \\ = \quad ? \quad * \quad ? \end{array}$$

Árbol de recursión: ¿Cuánto cuesta cada nivel?

Para calcular el costo total de un nivel k , se deben multiplicar dos factores:

1. Número de nodos en el nivel k

Si en cada paso el problema se divide en a subproblemas, en el nivel k habrá:

$$\text{Núm. de nodos} = a^k$$

Nivel	Num. nodos
0	1
1	a^1
2	a^2
...	...

2. Costo de cada nodo individual

Si el tamaño original n se divide por b en cada nivel, el tamaño del problema en el nivel k es n/b^k . Por lo tanto, el trabajo realizado por un solo nodo es:

$$\text{Costo de un nodo} = f(n/b^k)$$

$$\text{Costo del nivel } i = (\text{Número de nodos}) * (\text{Costo del nodo})$$

$$C_i = a^k * f(n/b^k)$$

Árbol de recursión: ¿Cuántos niveles hay?

¿Cuándo termina la recursión?

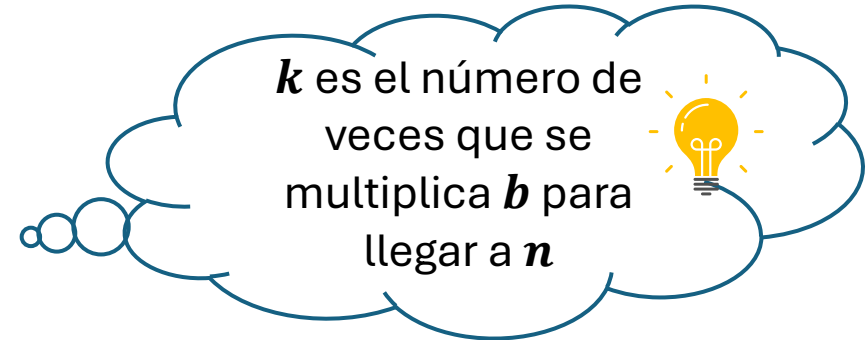
La recursión termina cuando el tamaño llega a 1:

$$\frac{n}{b^k} = 1$$

Ese ***k*** será el número de niveles.

Despejando ***k*** :

$$n = b^k$$



Aplicando **logaritmos**:

$$k = \log_b n$$

$$\text{Número de niveles (k)} = \log_b n + 1$$

El árbol tiene **$\log_b n + 1$** niveles (contando el nivel del nodo raíz)

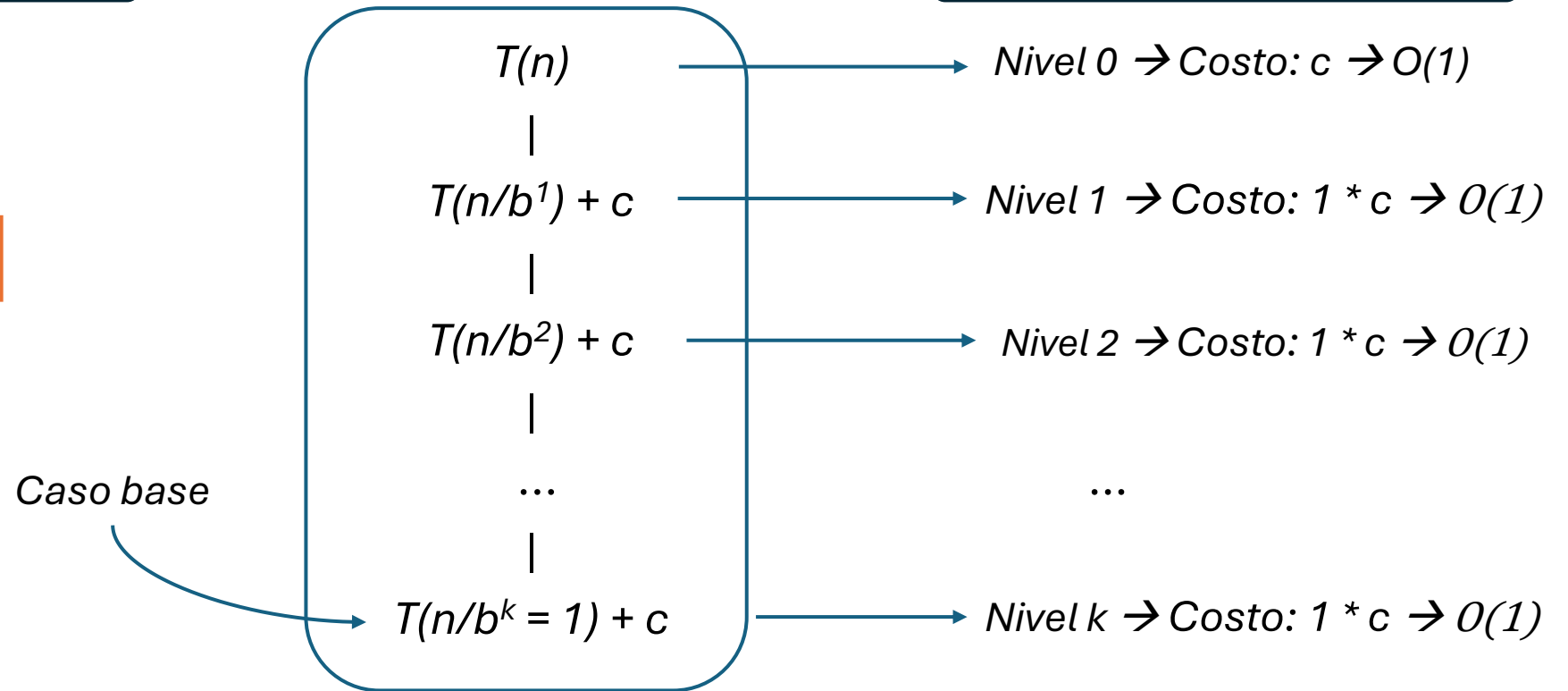
Árbol de recursión: Ejemplo genérico

$$T(n) = T\left(\frac{n}{b}\right) + c$$

¿Cuántos k niveles hay?

$$k = \log_b n + 1$$

¿Cuánto cuesta cada nivel?



Costo total = Costo por nivel * número de niveles

$$= (1 * c) * \log_b n + 1$$

$$= c * \log_b n + 1$$

$$O(\log_b n)$$

Árbol de recursión: Ejemplo 1

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

- **Nivel 0 (Raíz):**
 - 1 nodo de costo n
- **Nivel 1:**
 - 2 nodos de tamaño $n/2$.
 - Costo por nodo es $n/2$.
 - El costo total del nivel es $2 * (n/2) = n$.
- **Nivel 2:**
 - 4 subproblemas de tamaño $n/4$.
 - Costo por nodo es $n/4$
 - El costo total del nivel es $4 * (n/4) = n$
- **Nivel 3:**
 - 8 subproblemas de tamaño $n/8$
 - Costo por nodo es $n/8$
 - El costo total del nivel es $8 * (n/8) = n$

Árbol de recursión: Ejemplo 1

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Nivel	Número de nodos	Tamaño del subproblema	Costo por nodo	Costo total por nivel
0	$2^0 = 1$	n	n	n
1	$2^1 = 2$	$n/2$	$n/2$	$2 * (n/2) = n$
2	$2^2 = 4$	$n/4$	$n/4$	$4 * (n/4) = n$
3	$2^3 = 8$	$n/8$	$n/8$	$8 * (n/8) = n$
...	...	...	...	...
k	2^k	$n/2^k$	$n/2^k$	$2^k * \left(\frac{n}{2^k}\right) = n$

Costo por nivel = n

Núm. Niveles = $\log_2 n + 1$

Costo total = costo por nivel * núm. Niveles

$$= n * (\log_2 n + 1)$$

$$= n \log_2 n + n$$

Complejidad: $O(n \log n)$

El costo es **igual** en cada nivel



Árbol de recursión: Ejemplo 2

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

- **Nivel 0 (Raíz):**
 - 1 nodo de costo n
- **Nivel 1:**
 - 4 subproblemas (nodos) de tamaño $n/2$.
 - Costo por nodo es $n/2$.
 - El costo total del nivel es $4 * (n/2) = 2n$.
- **Nivel 2:**
 - 16 subproblemas de tamaño $n/4$.
 - Costo por nodo es $n/4$
 - El costo total del nivel es $16 * (n/4) = 4n$
- **Nivel 3:**
 - 64 subproblemas de tamaño $n/8$
 - Costo por nodo es $n/8$
 - El costo total del nivel es $64 * (n/8) = 8n$

Árbol de recursión: Ejemplo 2

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

Nivel	Número de nodos	Tamaño del subproblema	Costo por nodo	Costo total por nivel
0	$4^0 = 1$	n	n	n
1	$4^1 = 4$	$n/2$	$n/2$	$4 * (n/2) = 2n$
2	$4^2 = 16$	$n/4$	$n/4$	$16 * (n/4) = 4n$
3	$4^3 = 64$	$n/8$	$n/8$	$64 * (n/8) = 8n$
...	...	...	...	...
k	4^k	$n/2^k$	$n/2^k$	$4^k * \left(\frac{n}{2^k}\right) = 2^k n$

El costo es **diferente** en cada nivel

Aplicando la serie geométrica:

$$n * (1 + 2 + 4 + 8 + \dots + 2^{\log_2 n})$$

$$\text{Costo total} = n * \left(\frac{r^{\text{niveles}-1}}{r-1}\right), r=2$$

$$= n * \left(\frac{2^{\log_2 n + 1} - 1}{2 - 1}\right)$$

$$= n * ((2^{\log_2 n} * 2^1) - 1)$$

$$= n * ((n * 2) - 1)$$

$$= n * (2n - 1)$$

$$\text{Costo total} = 2n^2 - n$$

$$\text{Complejidad: } O(n^2)$$

Árbol de recursión: Ejemplo 3

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

- **Nivel 0 (Raíz):**
 - 1 nodo de costo n^2
- **Nivel 1:**
 - 2 nodos de tamaño $n/2$.
 - Costo por nodo es $(n/2)^2 = n^2/4$
 - El costo total del nivel es $2 * (n^2/4) = n^2/2$.
- **Nivel 2:**
 - 4 subproblemas de tamaño $n/4$.
 - Costo por nodo es $(n/4)^2 = n^2/16$
 - El costo total del nivel es $4 * (n^2/16) = n^2/4$
- **Nivel 3:**
 - 8 subproblemas de tamaño $n/8$
 - Costo por nodo es $(n/8)^2 = n^2/64$
 - El costo total del nivel es $8 * (n^2/64) = n^2/8$

Árbol de recursión: Ejemplo 3

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

Nivel	Número de nodos	Tamaño del subproblema	Costo por nodo	Costo total por nivel
0	$2^0 = 1$	n	n^2	$1 * n^2 = n^2$
1	$2^1 = 2$	n/2	$(n/2)^2 = n^2/4$	$2 * (n^2/4) = n^2/2$
2	$2^2 = 4$	n/4	$(n/4)^2 = n^2/16$	$4 * (n^2/16) = n^2/4$
3	$2^3 = 8$	n/8	$(n/8)^2 = n^2/64$	$8 * (n^2/64) = n^2/8$
...	...	...	...	...
k	2^k	$n/2^k$	$(n/2^k)^2$	$2^k * (n/2^k)^2$

El costo es **diferente** en cada nivel

Aplicando la serie geométrica:

$$n^2 \left(1 + \frac{1}{2} + \frac{1}{4} + \dots \right)$$

$$\text{Costo total} = n^2 * \left(\frac{r^{\text{niveles}-1}}{r-1} \right), r = \frac{1}{2}$$

$$= n^2 * \left(\frac{(1/2)^{\log_2 n + 1} - 1}{(1/2) - 1} \right) *$$

$$= 2n^2 - 2n$$

Usando la paradoja de Zenón:

$$\left(1 + \frac{1}{2} + \frac{1}{4} + \dots \right) \text{ nunca va a pasar de } 2$$

$$= n^2 * 2$$

$$\text{Costo total} = 2n^2 - \text{algo pequeño}$$

$$\text{Complejidad: } O(n^2)$$